C Programming

中山大学专业课《程序设计I》

第8节 数组与指针

岳 晟

网络空间安全学院 工学园2栋508

yuesh5@mail.sysu.edu.cn

2025年秋季

复习：多维数组

如何理解多维数组初始化?

将多维数组理解为特殊的“一维数组”



复习：多维数组

```
int a[3][4]={{1},{5,6}};
```

复习：多维数组

a是包含3个元素的“一维数组”

```
int a[3][4]={{1},{5,6}};
```

复习：多维数组

最外层赋值

```
int a[3][4]={{1},{5,6}};
```

外层赋值是告诉编译器：**需要给a的哪些元素赋值**

int **a[3]**[4]={ { }, { } };

↓ a[0]　　↓ a[1]

但a包含3个元素，因此第3个元素**a[2]会被初始化0**

复习：多维数组

内层赋值

```
int a[3][4]={{1},{5,6}};
```

内层赋值是告诉编译器：**元素具体的值**

```
int a[3][4]={ {1}, {      } };
```

↓

a[0]={1} ⟶ **a[0][0]=1**
a[0][1]=0
a[0][2]=0
a[0][3]=0

复习：多维数组

内层赋值

```
int a[3][4]={{1},{5,6}};
```

内层赋值是告诉编译器：**元素具体的值**

```
int a[3][4]={ {1}, {5,6} };
```

↓

a[1]={5,6} ⟶ **a[1][0]=5**
a[1][1]=6
a[1][2]=0
a[1][3]=0

复习：多维数组

```
int a[3][4]={{1},{5,6}};
```

|||

```
int a[3][4]={ {1,0,0,0}, {5,6,0,0}, {0,0,0,0} };
```

```
int a[][4]={{1},{5,6}};
```

为什么初始化多维数组时不能省略其他维度？

一维数组与函数

如何将一维数组作为参数传入某个函数?

例如：函数实现一维数组加法

```
void vadd(int a[], int b[], int size){
    for (int i=0; i<size; i++)
        a[i] += b[i];
}
```

不定长一维数组

(若定义为a[10]，编译器将忽略数组长度)

```
...
vadd(a, b, size); // 调用时不需要加[]
...
```

多维数组与函数

如何将多维数组作为参数传入某个函数?

例如： 函数实现3×3矩阵加法

```
void matrix_add(int a[][3], int b[3][3],  int sz){
   for (int i=0; i<sz; i++)
              for (int j=0; j<3; j++)
                   a[i][j] += b[i][j];
}
```

除第一维外，
其他维长度必须包含

b[3][3]将被编译器解释为b[][3]

```
...
matrix_add(a, b, size); // 调用时不需要加[]
...
```

为什么函数可以改变原数组的值?

给数组形参变量传递的是首元素地址！

如何保护main中的数据?

```
/*vector add*/
#include<stdio.h>
#define  LEN_ARR(a) (sizeof(a) / sizeof(a[0]))
void vadd(int a[const], const int b[], const size_t sz);

int main(void){
   int a[9] = {1,2,3,4,5,6,7,8,9};
   vadd(a,a,LEN_ARR(a));
   for (int i=0;i<LEN_ARR(a);i++)
      printf("%d,",a[i]);
   printf("\n");
}
void vadd(int a[const], const int b[], const size_t sz){
   for (int i=0;i<sz;i++) {
      a[i] += b[i];
      }
}
```

const 在函数申明中应用

- const int b[] 表示 **b的元素** 不可修改
- int a[const] 表示 **形参a** 不可修改
- const size_t sz 表示 **sz** 不可修改

size_t 无符号整型的别名

- 函数中无法获得数组的长度，通常会将数组长度作为参数传给函数，类型常用 size_t

思考

用以下方式实现矩阵相加可不可行？为什么？

```
// a被视为一个二维数组数组首元素的地址
void matrix_add(int a[][3], int b[][3], size_t sz){
    vadd(a, b, sz*LEN_ARR(a[0]));
}
```

```
// a被视为一个一维数组首元素的地址
void vadd(int a[], int b[], int size){
    for (int i=0; i<size; i++)
        a[i] += b[i];
}
```

思考题

请运行 8_1_matrix_add.c，思考原因

什么是指针?

指针是存放内存地址的变量

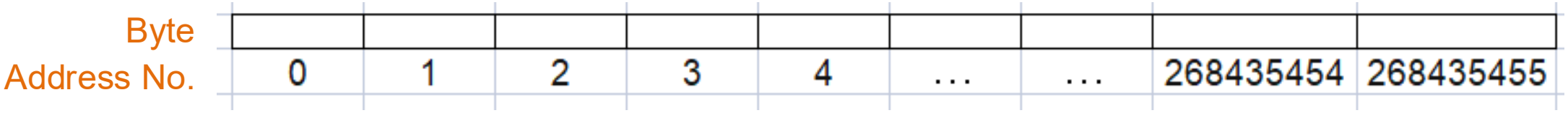
计算机的内存

- 计算机的内存在物理上是一组DRAM (动态随机存取存储器) 芯片
- 内存采用**字节 (byte) 编址**，即每个地址存8个位 (bit)



内存逻辑模型与定义

- 我们无需了解内存的物理结构——操作系统将硬、软件结合起来，为每个**进程** (运行的程序) 提供了**内存抽象 (逻辑) 模型**



- 逻辑内存可看作一个线性的字节数组，每一个**字节**都是由**8位二进制**组成，有**唯一编号**
- 编号**从0开始**直到**最后一个字节** (上图是一个256M的内存，一共有 2^{28} = 268435456个字节，它的地址范围就是 0 ~268435455)
- **地址**就是每一个**字节的编号**
- 程序中的变量、字面量、函数等数据载入内存中后，都有唯一的地址

指针变量

数据对象第一个字节的地址

- 指针是存放**指定类型数据**内存地址的**变量**
- 指针变量声明与定义的一般形式为：

变量类型 *指针变量名[=初始地址]

如：

```
int    *p1;        /* 整型指针 */
double *p2;        /* 双精度浮点型指针 */
char   *p3;        /* 字符型指针*/
```

注意：

1. 声明语句中，**星号** (*) 是用来指定一个变量是指针
2. 指针的**值**是32位或64位整数作为地址，可以使用 **%p** 来打印指针的值
3. 指针的**变量类型**，表示指针所存内存地址上存放的数据类型
4. 指针变量可以使用使用 extern、static、auto、register **存储类型修饰**

NULL 空指针常量

- NULL 是一个定义在标准库中的**值为零的常量**
- 指针声明时，若没有合适的地址初始化，则可以为它赋NULL表示当前**指针空闲**

```
/*pointer NULL*/
#include<stdio.h>

int main(void) {
    int a = 10;
    int *p = NULL;
    if (p) printf("*p=%d\n",*p);
    else printf("NULL pointer!");
    return 0;
}
```

指针运算符

- 取址运算符 **&**：**&左值表达式** 可得到该表达式对象的地址值，如：
  - int *intp = &i;
- 解引用运算符 ***：***指针** 代表指针/地址所指向的对象，如：
  - *intp = 3； // 作为左值表达式
  - j=*intp； // 作为右值表达式

问题

若 int a = 0; int *p = &a;

那么 &*p 的值是？ &*a 的值是？ *&a 的值是？

指针运算符

```
/*pointer ops*/
#include<stdio.h>

int main(void) {
    int a = 0;
    int *p = &a;
    *p = 10;  //对变量a赋值10
    printf("pointer p=%p, x=%x\n", p, p);
    printf("address &a=%p, &*p=%p\n", &a, &*p);
    printf("value *p=%d, a=%d\n", *p, a);
    return 0;
}
```

- *p 和 a 有相同的地址和值，*p 称为 a 的**引用/别名**
- * 运算将指针的值 (地址) 解析为指定类型对象或变量的引用

指针的算术、比较运算

- 指针类型数据可采用四种算术运算：+、-、++、--和**比较运算符**
- 给定 int *p, *q, i; 指针算术、比较运算如下表

	语法	说明	等价
算术运算	q - p	p、q指针之间 int 元素个数	(addr_q-addr_p) / sizeof(int)
	p++ 或 ++p	p 指向下一个 int 元素	
	p-- 或 --p	p 指向上一个 int 元素	
	p + i	p 指向后第 i 个 int 元素	addr_p + i*sizeof(int)
	p - i	p 指向前 i 个 int 元素	addr_p - i*sizeof(int)
比较运算	p==q 或 p!=q	p、q 是否有相同地址	
	p>q 或 p>=q	p 中地址是否高于 q 中地址	
	p<q 或 p<=q	p 中地址是否低于 q 中地址	

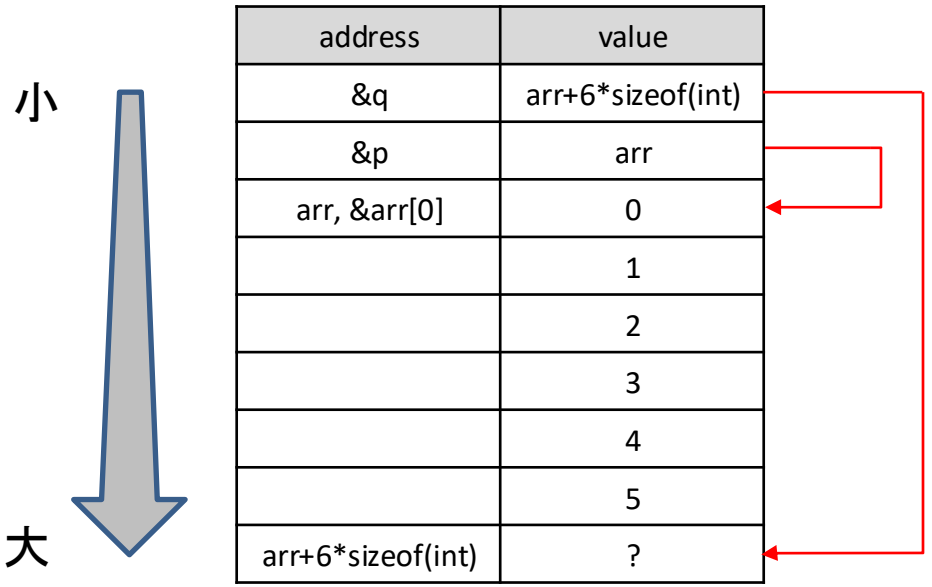
思考：printf会输出什么?

```
/*pointer-arithmetic-ops */
#include<stdio.h>

int main(void) {
    int arr[] = {0,1,2,3,4,5};
    int *p = arr;
    int *q = &arr[sizeof(arr)/sizeof(int)];
    printf("q-p=%d\n", q-p); // ①
    for (; p<q; p++)
        printf("%d", *p);    // ②
    printf("\n");
    for (q=q-1; q>=arr; q--)
        printf("%d", *q);    // ③
    printf("\n");
    return 0;
}
```

```
q-p=6
012345
543210
```

变量申明后的内存模型



指针与数组：数组下标运算

- arr[i] 等价于 *(arr + i)
- 运行右边程序
  - p 指向 a 的首元素 a[0]
  - p[i] 和 a[i] 可访问相同的元素
  - p[i] 等价于 *(p+i)

```
/*pointer index ops*/
#include <stdio.h>
int main(void)
{
    int a[3] = {1,2,3};
    int *p = a;
    printf("a[2]=%d *(a+2)=%d\n", a[2], *(a+2));
    printf("p[2]=%d *(p+2)=%d\n", p[2], *(p+2));

}
```

指针与数组：数组下标运算

- arr[i] 等价于 *(arr + i)
- 运行右边程序
  - p 指向 a 的首元素 a[0]
  - p[i] 和 a[i] 可访问相同的元素
  - p[i] 等价于 *(p+i)
- 数组下标运算符拥有形式
  - 指针表达式 [整数表达式]
  - 整数表达式 [指针表达式]

```
/*pointer index ops*/
#include <stdio.h>
int main(void)
{
    int a[3] = {1,2,3};
    int *p = a;
    printf("a[2]=%d *(a+2)=%d\n", a[2], *(a+2));
    printf("p[2]=%d *(p+2)=%d\n", p[2], *(p+2));

    printf("%d %d\n", a[2], 2[a]);
    2[a] = 7;
    printf("%d %d\n", p[2], 2[p]);
}
```

指针与数组：联系与区别

- 对于 type *p 和 type a[N]，下表显示了它们的区别与联系

指针 p	数组 a	备注
下标运算 p[i]	下标运算 a[i]	设 p == a, p[i] 和 a[i] 是同一对象
算术运算 p + i	算术运算 a + i	设 p == a, p+i 和 a+i 是相同指针
解引用 *p	解引用 *a	设 p == a, 解引用对象是 a[0]
p 是变量（左值）	a 是地址值（右值）	p = a 是正确，**a = p 错误**
自增减和赋值运算	**不允许**	
取地址运算	--	
--	可定义并分配连续的空间	
可作为函数形参	作为函数实参传入时发生衰退 (decay)	**函数形参声明中 *p 等价于 p[]**，但 p[] 不能用于数组定义
sizeof(p) 返回指针字节数	sizeof(a) 返回a数组空间字节数	p 不具备数组类型特征，即无法从 p 获得数组元素数量

指针与数组：指针的数组

```
/*array of pointers*/
#include<stdio.h>
#define N 3

int main(void) {
    int arr[N] = {1,2,3};
    int *p[N] = {};   //[] () 运算符高于其他一元运算符
                      //解释为 int * 类型的数组
    for(int i=0; i<N; i++)
        p[i] = &arr[i];
    for(int i=0; i<N; i++)
        printf("*p[%d]=%d\n", i, *p[i]);

    return 0;
}
```

🙋‍♂有同学说，p 指向一个三角矩阵：

{{1，2，3},

{2，3},

{3}}

对吗?

p[i][j] = *(p[i] + j)

= *(&a[i] + j)

= a[i+j]

指向数组的指针

```
/*pointer to array*/
#include<stdio.h>
#define N 3

int main(void) {
    int arr[][N] = {{1,2,3}, {2,3}, {3}};
    int (*q)[N] = arr;     //注意用括号，区别指针的数组
                           //解释为 int[] 类型的指针
    for(int i=0; i<N; i++) {
//        printf("*q[%d] = %d\n", i, *q[i]);
        for(int j=0; j<N-i; j++)
            printf("%4d", q[i][j]);
        printf("\n");
    }
    return 0;
}
```

q指向**大小为N的一维数组**

等等？？？？？？

***p[N], (*q)[N] 都能用于表达二维数组，它们的区别是?**

指向指针的指针

```c
/*pointer to pointer*/
#include<stdio.h>

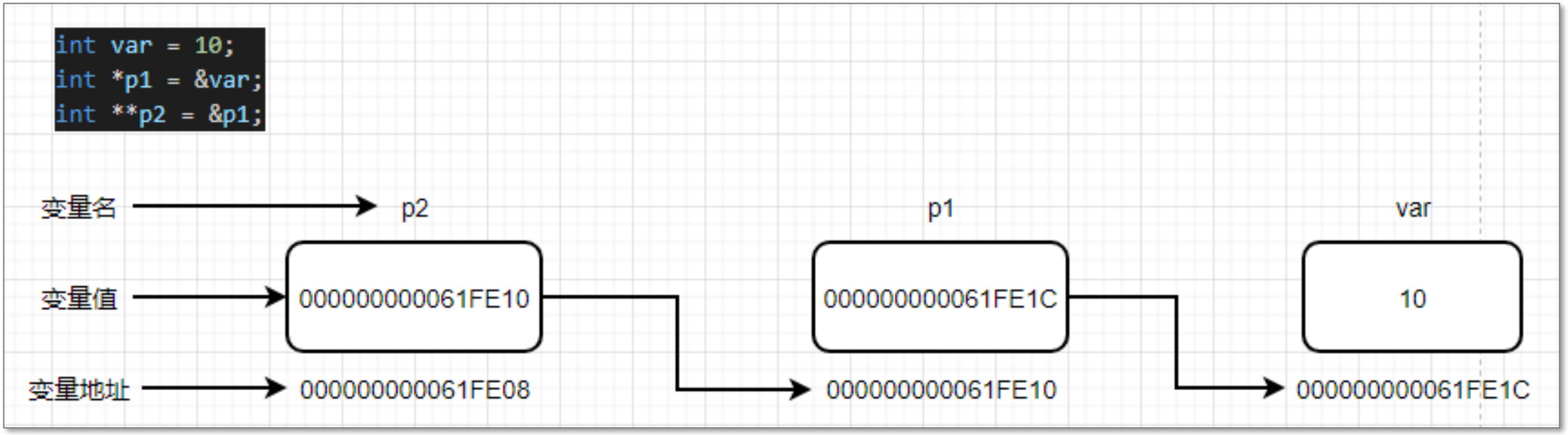
int main(void) {
    int var = 10;
    int *p1 = &var;
    int **p2 = &p1;
    printf("p1=%p\n", p1);
    printf("*p1=%d\n", *p1);
    printf("p2=%p\n", p2);
    printf("*p2=%p\n", *p2);
    printf("**p2=%d\n", **p2);

    return 0;
}
```

p2是**指向指针的指针**，q的值是一个地址

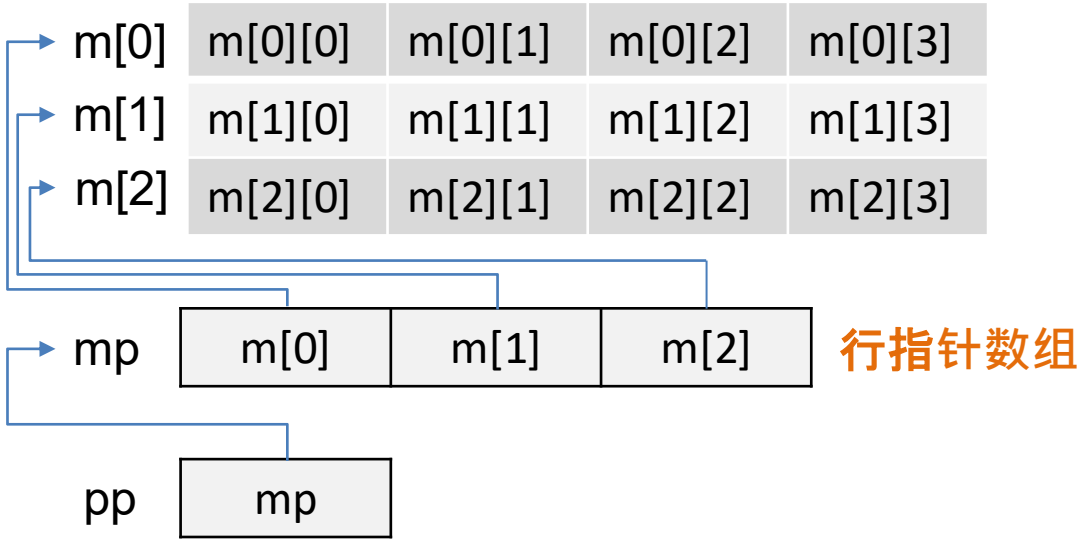
指向指针的指针

- 当一个变量被一个指针指向的指针指向时，访问这个值需要使用两个星号运算符，即先解引用到指针变量，再解引用到变量



指向数组的指针 vs 指针的数组 vs 指向指针的指针

- int m[3][4] 的存储结构如图所示：



- 无论p、mp或pp都可以正确打印二维数组！

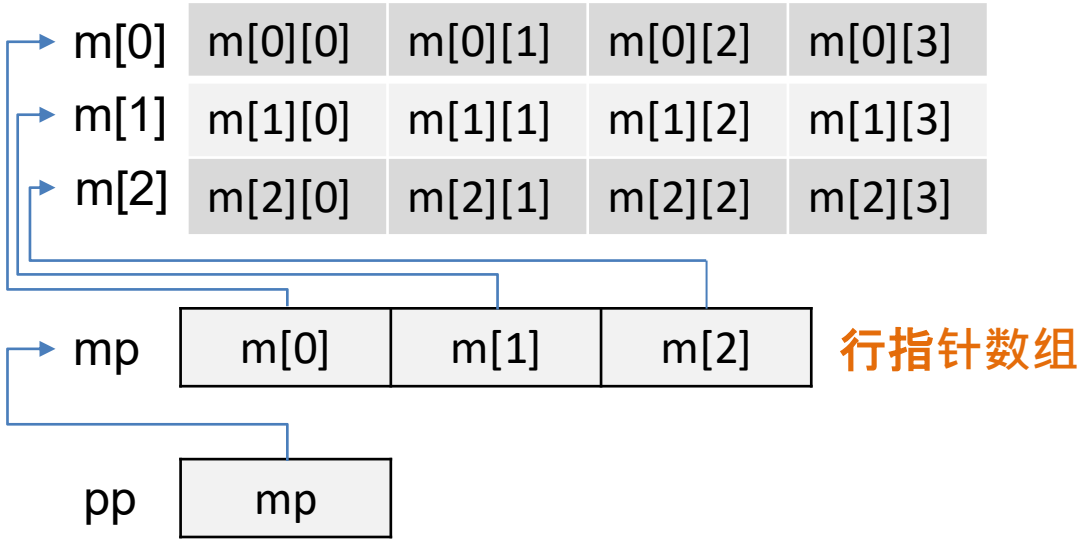
```c
/*pointer array*/
#include<stdio.h>

int main(void) {
    int m[3][4] = {1,2,3,4,5,6,7,8,9,10,11,12};
    int (*p)[4] = m; //指向数组的指针
    for (int i=0; i<3; i++)
        printf("%p, %p\n", p[i], m[i]);
    int *(mp[3])      //(指向数组的)指针的数组
        = {m[0],m[1],m[2]};
    int **pp = mp;    //(指向)指针(数组)的指针
    for (int i=0; i<3; i++) {
        for (int j=0; j<4; j++)
            printf("%4d", pp[i][j]);
        printf("\n");
    }
    return 0;
}
```

程序会输出什么?

练习：数组的指针 vs 指针的数组 vs 指针的指针

- int m[3][4] 的存储结构如图所示：



- 无论p、mp或pp都可以正确打印二维数组！

```c
/*pointer array*/
#include<stdio.h>

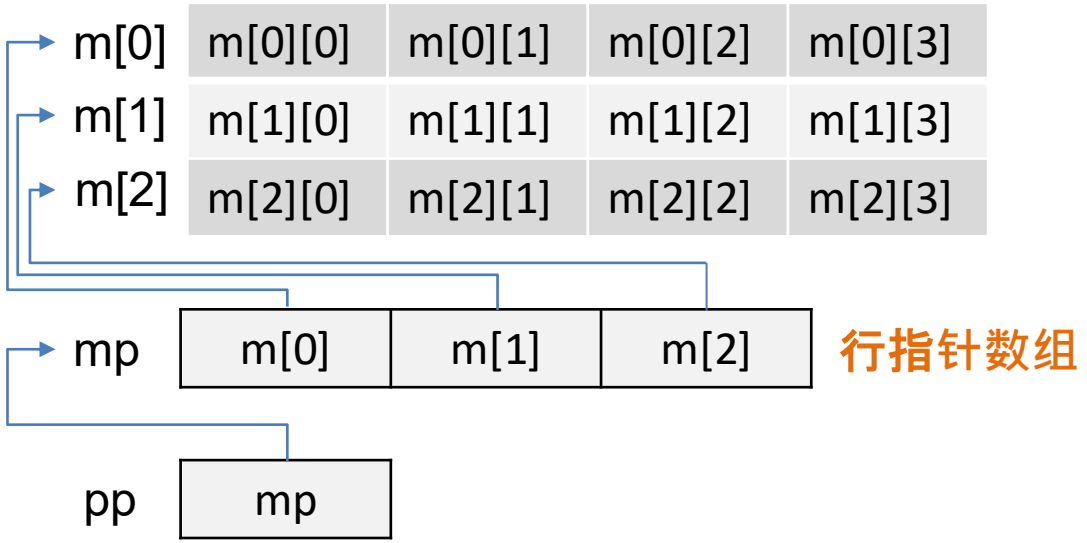
int main(void) {
    int m[3][4] = {1,2,3,4,5,6,7,8,9,10,11,12};
    int (*p)[4] = m; // 指向数组的指针
    int *(mp[3])     // (指向数组的)指针的数组
        = {m[0],m[1],m[2]};
    int **pp = mp;   // (指向)指针(数组)的指针

    // 请练习：
    // 分别使用 m,p,mp,pp 访问元素 7
    // 根据左图，思考计算过程
    // 并写出访问元素 m[0][0]的几种方法

    return 0;
}
```

二维数组的表示方法

- int m[3][4] 的存储结构如图所示：



- 无论p、mp或pp都可以正确打印二维数组！

- 声明存储空间：
  - int m[3][4];
- 未知长度数组 (必须是连续空间)：
  - int m[][4]; //用于函数形参
  - int (*p)[4]; //数组的指针
- 拥有行指针数组的矩阵 (行之间不要求连续空间，每行长度也可不一样)：
  - int *(mp[3]); //指针的数组
  - int **pp; //指针的指针

指针与函数参数：引用变量

C语言允许传递指针 (地址) 给函数，只需声明**函数参数为指针类型**即可

```
#include <stdio.h>

void cswap(int *a, int *b) {
    int m = *a;
    *a = *b;
    *b = m;
}

int main() {
    int a = 3, b = 4;
    printf("a = %d, and b = %d\n", a, b);
    cswap(&a, &b);
    printf("a = %d, and b = %d\n", a, b);
}
```

利用指针函数可以直接**操作**指针指向的数据

指针与函数参数：引用数组

同样地， 也可以通过指针参数访问和修改数组中的元素

```
#include <stdio.h>
#define MAXN 3

void inc(int *arr, int n) {
    for (int i = 0; i < n; i++)
        arr[i]++;
}

int main(void) {
    int arr[MAXN] = {1, 2, 3};
    inc(arr, MAXN);
    for (int i = 0; i < MAXN; i++)
        printf("arr[%d] = %d\n", i, arr[i]);

    return 0;
}
```

要点：

- 形参中 **int *arr** 等价于 **int arr[]**

问题：

- **T *p** 一定指向数组吗?
- **T **p** 等价于 **T p[][]** 吗?

指针与函数参数：引用多维数组

矩阵加法函数的形参如何用指针声明呢？

```
#include <stdio.h>

void matrix_add3(size_t m, size_t n, int (*a)[n], int (*b)[n]) {
    for (int i=0; i<m; i++)
        for (int j=0; j<n; j++)
            a[i][j] += b[i][j];
}

int main(void) {
    int m[3][4] = {1,2,3,4,5,6,7,8,9,10,11,12};
    matrix_add3(3, 4, m, m);
    for (int i=0; i<3; i++) {
        for (int j=0; j<4; j++)
            printf("%-4d",m[i][j]);
        printf("\n");
    }
}
```

地址总结

变量声明	变量类型	值
int a[N1];	int[N1]	a的值为首元素地址：&a[0]
int b[N1][N2];	int[N1][N2]	b的值为首元素地址：&b[0]
int *p1 = a;	int *	p1的值为&a[0]
int (*p2)[N2] = b;	int (*)[N2]	p2的值为&b[0]
int *p3[N3] = {a};	int *[N3]	p3的值为&p3[0]
int **p4 = p3;	int **	p4的值为p3（也就是&p3[0]）

作业

- 阅读《C程序设计》第8章
- 完成 PTA 本周习题

谢谢！

邮箱：yuesh5@mail.sysu.edu.cn